

INTERNNOVA – WEEK 6 FINAL PROJECT

Student Management System

Java Application Development

Final Project Documentation

1. Project Title

Student Management System

2. Project Description

Student Management System is a console-based Java application developed to manage student records efficiently. The application allows users to add, view, search, update, and delete student information. It also uses file handling to save and load student data and multithreading to perform a backup operation.

3. Objective

The objective of this project is to build a functional Java console-based management system and demonstrate object-oriented programming, exception handling, collections, file handling, interfaces, and multithreading.

4. Features

- Add student records
- View all student records
- Search student by Student ID
- Update student details
- Delete student records
- Save student data to a file
- Load student data from a file
- Backup student data using multithreading
- Input validation and exception handling
- Menu-driven console interface

5. Technologies Used

- Java
- Eclipse IDE
- Java Collections
- File Handling
- Git
- GitHub

6. Java Concepts Used

Classes and Objects: Used to create and manage Student, Person, and manager objects.

Constructors: Used to initialize objects when students are created.

this keyword: Used in constructors and methods to refer to the current object.

static: Used for maintaining the student count and shared scanner.

Encapsulation: Student and Person data are kept private and accessed through methods.

Inheritance: Student extends the abstract Person class.

Polymorphism: displayRole() is overridden in the Student class.

Abstraction: Person is an abstract class with an abstract displayRole() method.

Interface: Manageable defines the CRUD operations implemented by StudentManager.

Exception Handling: try-catch blocks handle invalid input and file-related errors.

ArrayList: Stores student records dynamically.

File Handling: students.txt is used to save and load student records.

Multithreading: BackupThread performs the backup operation in a separate thread.

7. Project Structure

```
StudentManagementSystem/  
├── src/  
│   └── studentmanagement/  
│       ├── Main.java  
│       ├── Person.java  
│       ├── Student.java  
│       ├── Manageable.java  
│       ├── StudentManager.java  
│       ├── FileHandler.java  
│       └── BackupThread.java  
├── students.txt  
├── screenshots/  
└── README.md
```

8. Source Code

8.1 Person.java

```
package studentmanagement;  
  
public abstract class Person {  
  
    private String name;  
    private int age;
```

```

public Person(String name, int age) {
    this.name = name;
    this.age = age;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public abstract void displayRole();
}

```

8.2 Student.java

```

package studentmanagement;

public class Student extends Person {

    private int studentId;
    private String course;
    private double marks;

    private static int studentCount = 0;

    public Student(int studentId, String name, String course, int age, double marks) {
        super(name, age);
        this.studentId = studentId;
        this.course = course;
        this.marks = marks;
        studentCount++;
    }

    public int getStudentId() {
        return studentId;
    }

    public String getCourse() {
        return course;
    }

    public double getMarks() {
        return marks;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    public void setMarks(double marks) {
        this.marks = marks;
    }

    public static int getStudentCount() {
        return studentCount;
    }
}

```

```

    }

    @Override
    public void displayRole() {
        System.out.println("Role: College Student");
    }

    public void displayStudent() {
        System.out.println("Student ID : " + studentId);
        System.out.println("Name      : " + getName());
        System.out.println("Course   : " + course);
        System.out.println("Age      : " + getAge());
        System.out.println("Marks   : " + marks);
    }
}

```

8.3 Manageable.java

```

package studentmanagement;

public interface Manageable {

    void addStudent(Student student);

    void viewStudents();

    void searchStudent(int studentId);

    void updateStudent(int studentId);

    void deleteStudent(int studentId);
}

```

8.4 StudentManager.java

```

package studentmanagement;

import java.util.ArrayList;

public class StudentManager implements Manageable {

    private ArrayList<Student> students = new ArrayList<>();

    @Override
    public void addStudent(Student student) {
        students.add(student);
        System.out.println("Student added successfully!");
    }

    @Override
    public void viewStudents() {

        if (students.isEmpty()) {
            System.out.println("No student records found.");
            return;
        }

        System.out.println("\n===== All Student Records =====");

        for (Student student : students) {
            student.displayStudent();
            System.out.println("-----");
        }

    }

    @Override
    public void searchStudent(int studentId) {

```

```

        for (Student student : students) {

            if (student.getStudentId() == studentId) {
                System.out.println("\nStudent Found!");
                student.displayStudent();
                return;
            }
        }

        System.out.println("Student not found.");
    }

    @Override
    public void updateStudent(int studentId) {

        for (Student student : students) {

            if (student.getStudentId() == studentId) {

                System.out.println("Student found.");

                System.out.print("Enter new course: ");
                String course = Main.scanner.nextLine();

                System.out.print("Enter new marks: ");
                double marks = Main.scanner.nextDouble();
                Main.scanner.nextLine();

                student.setCourse(course);
                student.setMarks(marks);

                System.out.println("Student updated successfully!");
                return;
            }
        }

        System.out.println("Student not found.");
    }

    @Override
    public void deleteStudent(int studentId) {

        for (Student student : students) {

            if (student.getStudentId() == studentId) {
                students.remove(student);
                System.out.println("Student deleted successfully!");
                return;
            }
        }

        System.out.println("Student not found.");
    }

    public ArrayList<Student> getStudents() {
        return students;
    }
}

```

8.5 FileHandler.java

```

package studentmanagement;

import java.io.BufferedReader;
import java.io.File;

```

```

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class FileHandler {

    private static final String FILE_NAME = "students.txt";

    public static void saveStudents(StudentManager manager) {

        try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_NAME))) {

            for (Student student : manager.getStudents()) {

                writer.println(
                    student.getStudentId() + "," +
                    student.getName() + "," +
                    student.getCourse() + "," +
                    student.getAge() + "," +
                    student.getMarks()
                );
            }

            System.out.println("Student data saved successfully.");

        } catch (IOException e) {
            System.out.println("Error while saving student data: " + e.getMessage());
        }
    }

    public static void loadStudents(StudentManager manager) {

        File file = new File(FILE_NAME);

        if (!file.exists()) {
            return;
        }

        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {

            String line;

            while ((line = reader.readLine()) != null) {

                String[] data = line.split(",");

                if (data.length == 5) {

                    int studentId = Integer.parseInt(data[0]);
                    String name = data[1];
                    String course = data[2];
                    int age = Integer.parseInt(data[3]);
                    double marks = Double.parseDouble(data[4]);

                    Student student = new Student(
                        studentId,
                        name,
                        course,
                        age,
                        marks
                    );

                    manager.getStudents().add(student);
                }
            }
        }
    }
}

```

```

        System.out.println("Student data loaded successfully.");

    } catch (IOException | NumberFormatException e) {
        System.out.println("Error while loading student data: " + e.getMessage());
    }
}
}

```

8.6 BackupThread.java

```

package studentmanagement;

public class BackupThread extends Thread {

    private StudentManager manager;

    public BackupThread(StudentManager manager) {
        this.manager = manager;
    }

    @Override
    public void run() {

        System.out.println("\nBackup process started...");

        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("Backup interrupted.");
        }

        FileHandler.saveStudents(manager);

        System.out.println("Backup process completed.");
    }
}

```

8.7 Main.java

```

package studentmanagement;

import java.util.Scanner;

public class Main {

    public static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {

        StudentManager manager = new StudentManager();

        FileHandler.loadStudents(manager);

        int choice;

        do {
            System.out.println("\n=====");
            System.out.println("      STUDENT MANAGEMENT SYSTEM");
            System.out.println("=====");
            System.out.println("1. Add Student");
            System.out.println("2. View All Students");
            System.out.println("3. Search Student");
            System.out.println("4. Update Student");
            System.out.println("5. Delete Student");
            System.out.println("6. Backup Data");
            System.out.println("7. Exit");

```

```

System.out.println("=====");

try {

    System.out.print("Enter your choice: ");
    choice = scanner.nextInt();
    scanner.nextLine();

    switch (choice) {

        case 1:
            addStudent(manager);
            break;

        case 2:
            manager.viewStudents();
            break;

        case 3:
            System.out.print("Enter Student ID to search: ");
            int searchId = scanner.nextInt();
            scanner.nextLine();

            manager.searchStudent(searchId);
            break;

        case 4:
            System.out.print("Enter Student ID to update: ");
            int updateId = scanner.nextInt();
            scanner.nextLine();

            manager.updateStudent(updateId);
            break;

        case 5:
            System.out.print("Enter Student ID to delete: ");
            int deleteId = scanner.nextInt();
            scanner.nextLine();

            manager.deleteStudent(deleteId);
            break;

        case 6:
            BackupThread backup = new BackupThread(manager);
            backup.start();
            break;

        case 7:
            FileHandler.saveStudents(manager);
            System.out.println("Thank you for using Student Management System!");
            break;

        default:
            System.out.println("Invalid choice. Please enter 1-7.");

    }

} catch (Exception e) {

    System.out.println("Invalid input. Please enter the correct value.");
    scanner.nextLine();
    choice = 0;

}

} while (choice != 7);

```



```

        scanner.close();
    }

    private static void addStudent(StudentManager manager) {

        try {

            System.out.print("Enter Student ID: ");
            int id = scanner.nextInt();
            scanner.nextLine();

            System.out.print("Enter Name: ");
            String name = scanner.nextLine();

            System.out.print("Enter Course: ");
            String course = scanner.nextLine();

            System.out.print("Enter Age: ");
            int age = scanner.nextInt();

            System.out.print("Enter Marks: ");
            double marks = scanner.nextDouble();
            scanner.nextLine();

            if (age <= 0 || marks < 0 || marks > 100) {
                System.out.println("Invalid age or marks.");
                return;
            }

            Student student = new Student(
                id,
                name,
                course,
                age,
                marks
            );

            manager.addStudent(student);

        } catch (Exception e) {

            System.out.println("Invalid input. Student was not added.");
            scanner.nextLine();
        }
    }
}

```

9. How to Run the Project

1. Install Java JDK.
2. Open Eclipse IDE.
3. Open/import the StudentManagementSystem project.
4. Open Main.java.
5. Run Main.java as a Java Application.
6. Select an option from the menu and enter the required student details.

10. Sample Output

```

=====
STUDENT MANAGEMENT SYSTEM

```

```
=====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Backup Data
7. Exit
=====
```

Example student record:

Student ID : 101

Name : Rahul

Course : Computer Science

Age : 20

Marks : 90.0

11. File Handling

The application stores student records in students.txt. FileHandler uses FileWriter and PrintWriter to save records and BufferedReader/FileReader to load records when the application starts.

12. Multithreading

BackupThread extends the Thread class. When the user selects Backup Data, a separate thread starts the backup operation. Thread.sleep() is used to demonstrate the threaded backup process.

13. Challenges and Solutions

- Handling invalid user input: try-catch blocks were used to prevent the application from stopping unexpectedly.
- Managing multiple student records: ArrayList was used to store student objects dynamically.
- Saving records permanently: file handling was implemented using students.txt.
- Demonstrating multithreading: BackupThread was created to perform the backup operation in a separate thread.

14. GitHub Repository

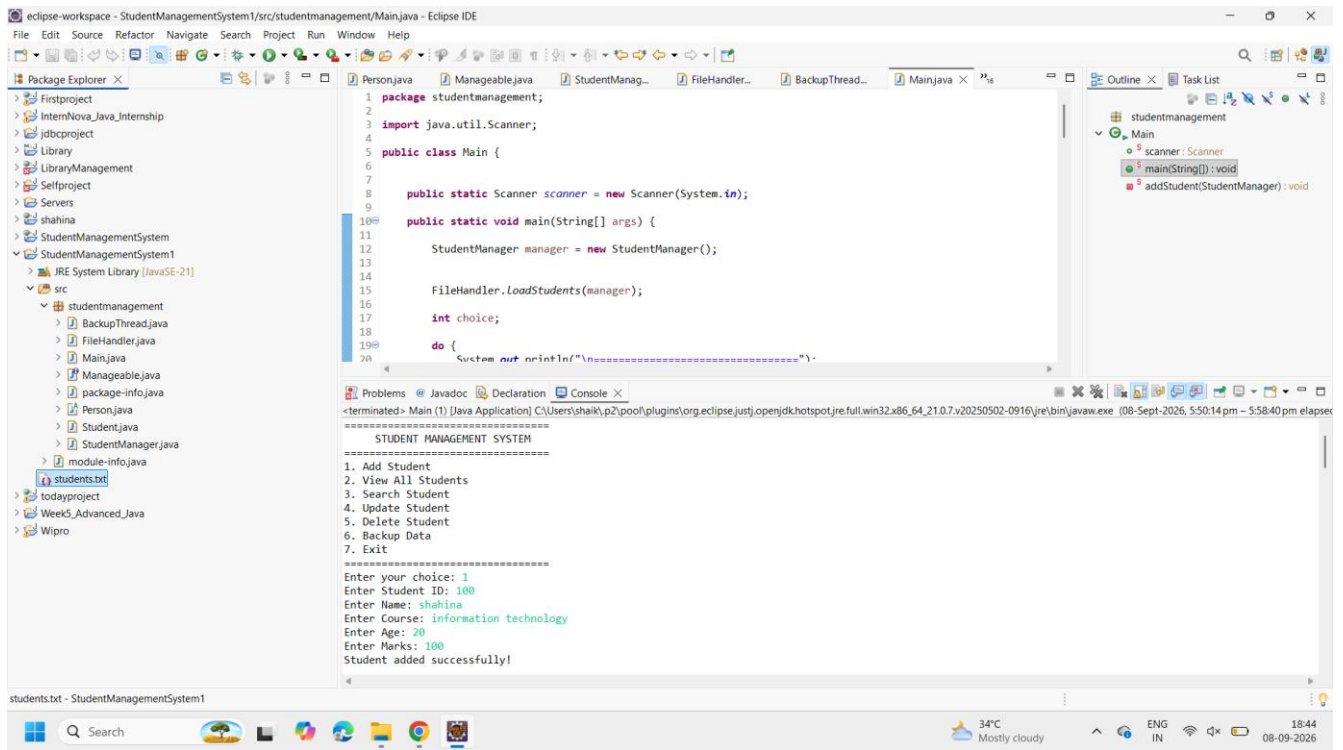
The complete source code, README.md, students.txt, and project screenshots are uploaded to the GitHub repository. Paste the actual GitHub repository link here before final submission.

GitHub Link: _____

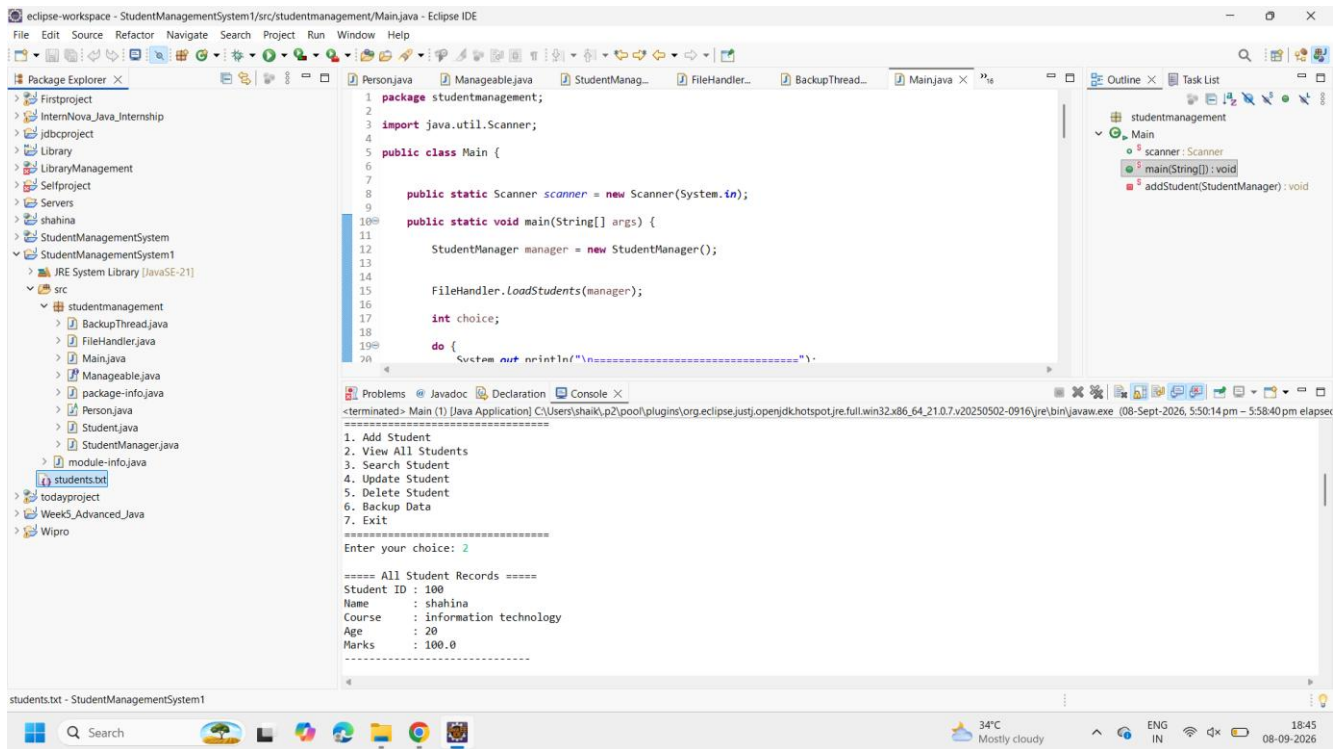
15. Screenshots

Insert the 7 project screenshots below. Recommended screenshots include source code and program output for the main features such as Add, View, Search, Update, Delete, Backup, and the main menu.

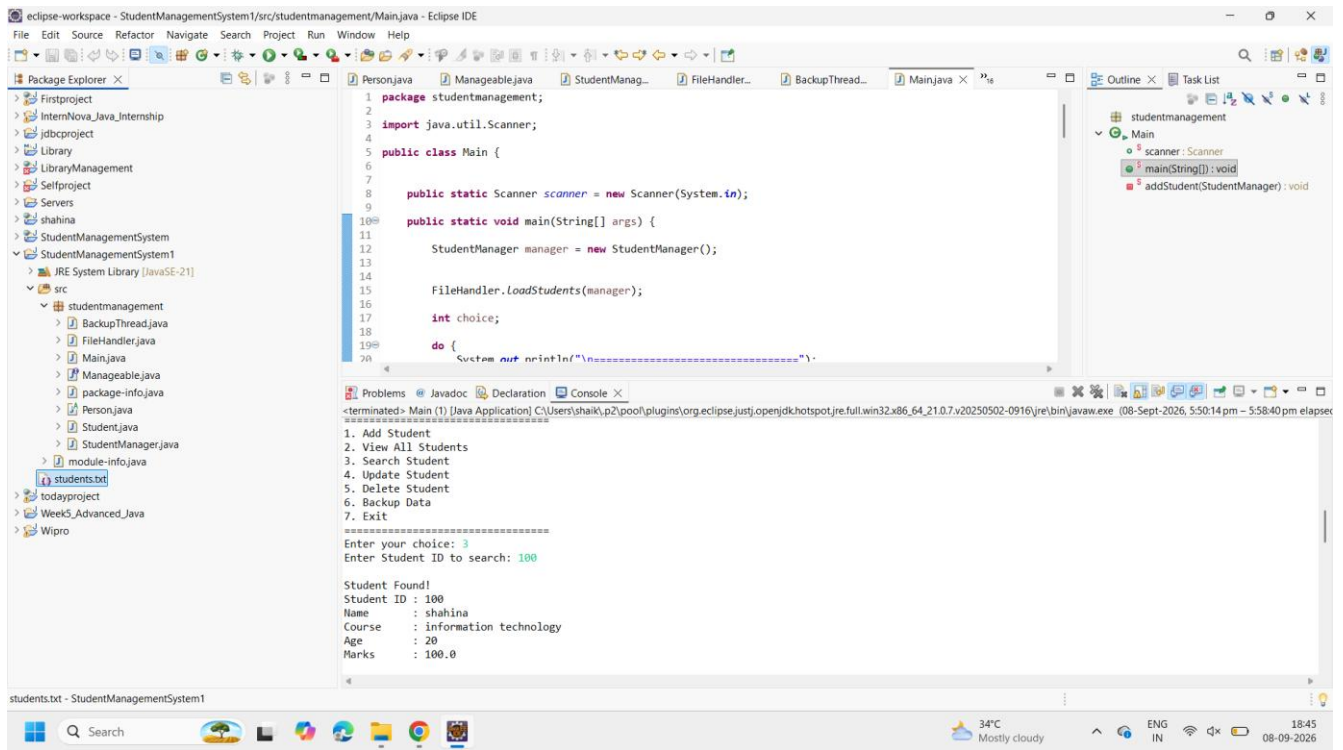
Screenshot 1:



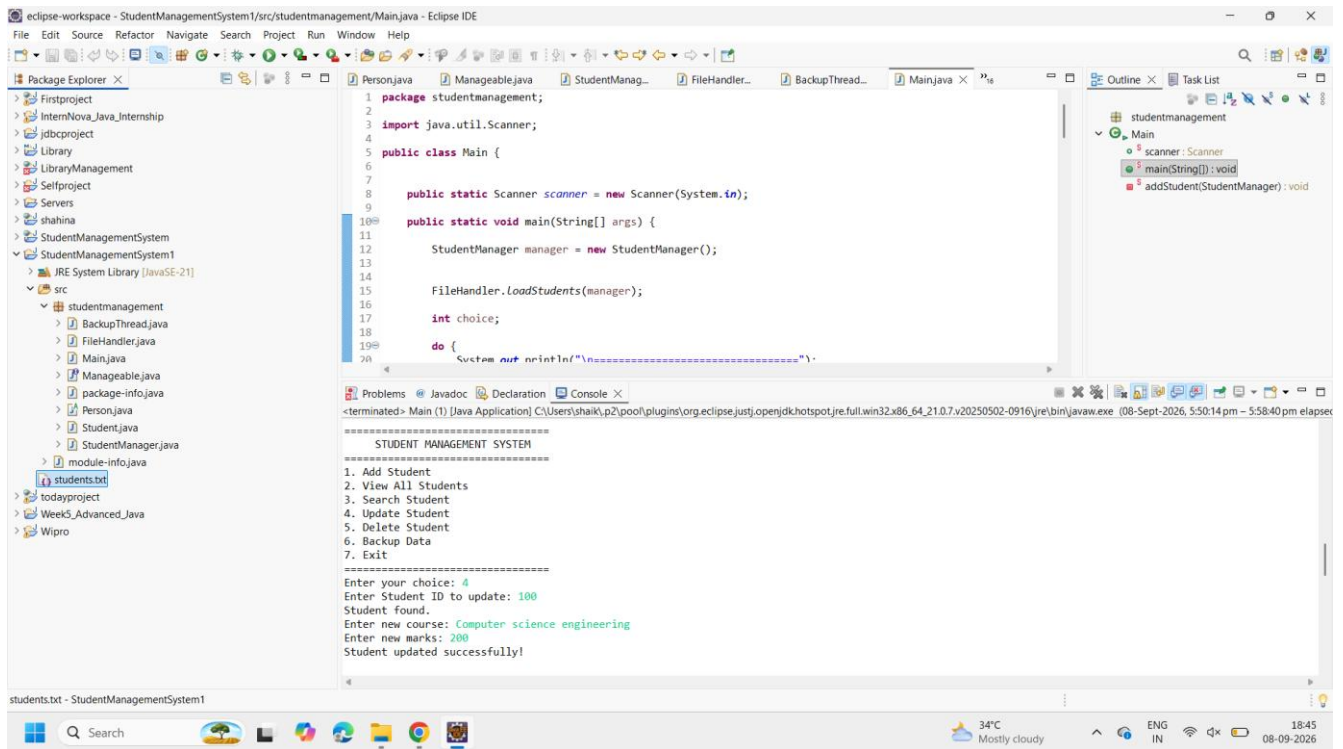
Screenshot 2:



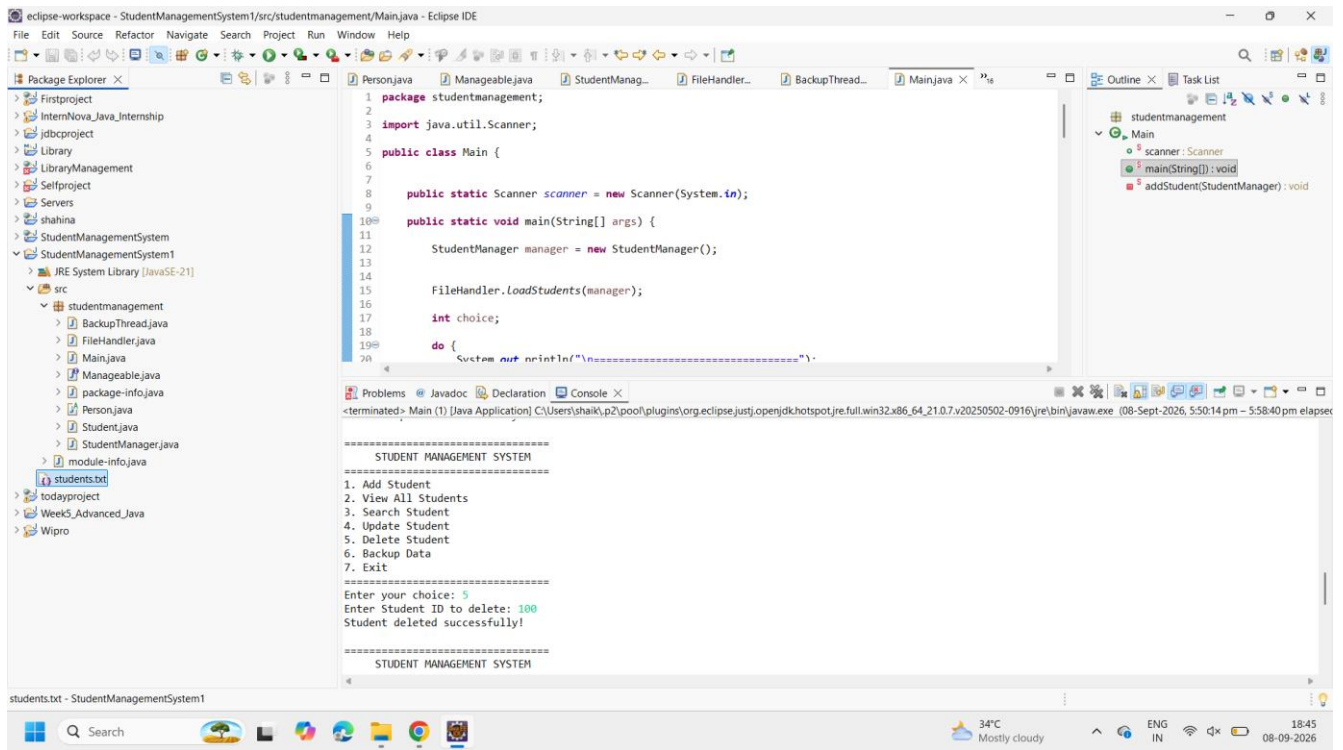
Screenshot 3:



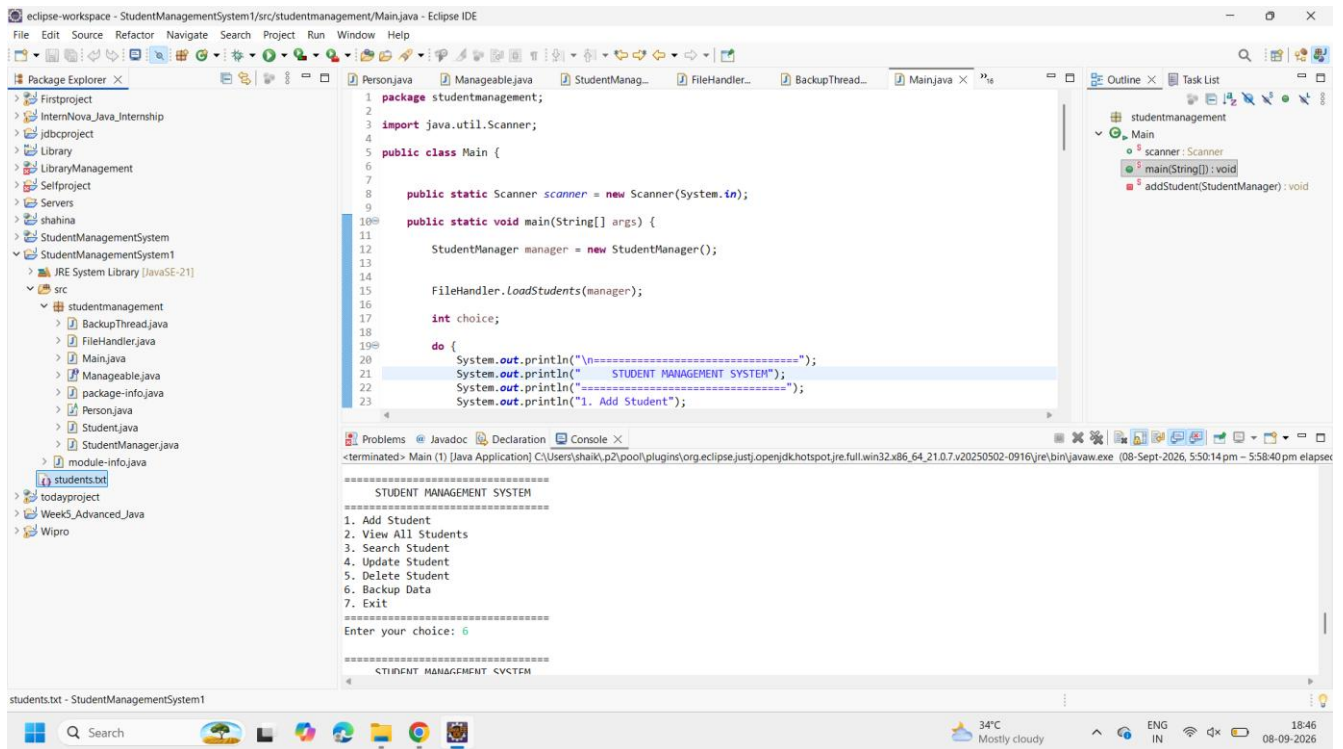
Screenshot 4:



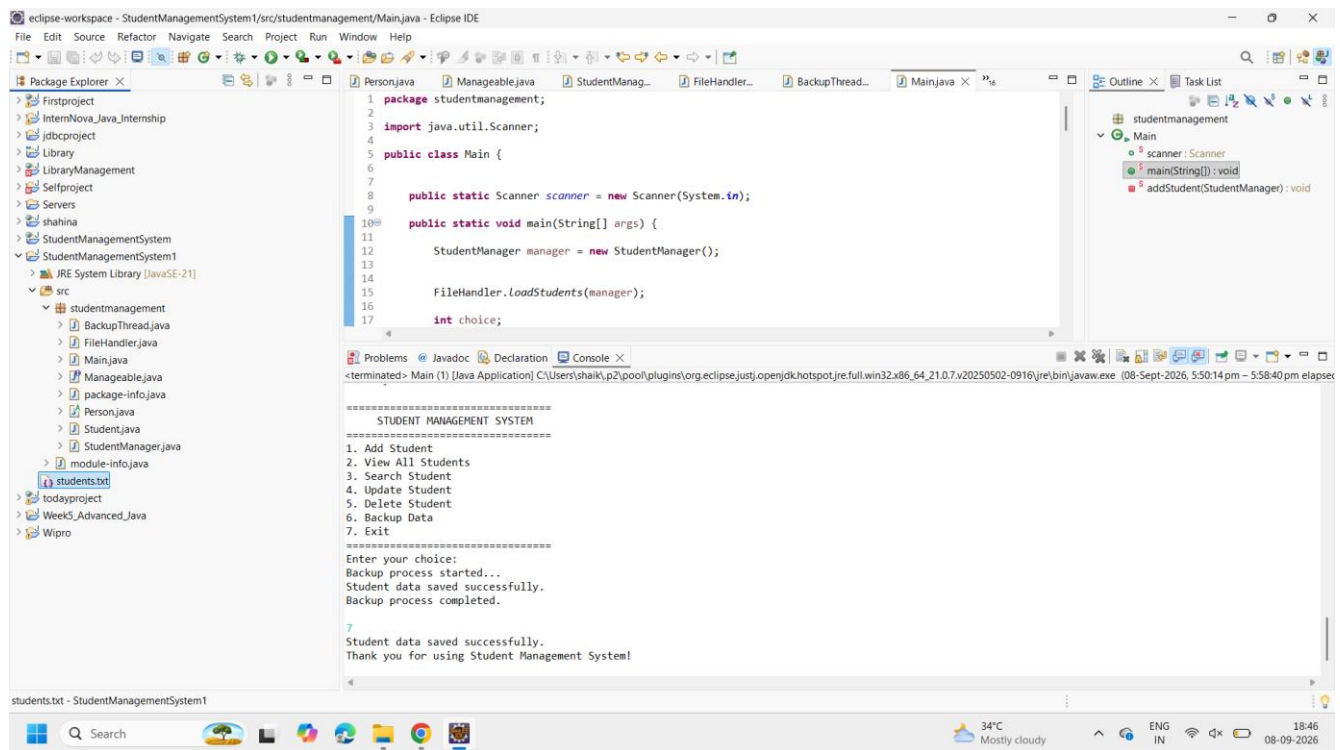
Screenshot 5:



Screenshot 6:



Screenshot 7:



16. Future Improvements

- Develop a graphical user interface.
- Add user login and authentication.
- Connect the application to a database using JDBC.
- Add sorting and filtering options.
- Generate student reports.
- Provide advanced search functionality.

17. Conclusion

The Student Management System successfully demonstrates the use of Java programming concepts in a practical console-based application. The project includes CRUD operations, object-oriented programming, exception handling, collections, file handling, an interface, and multithreading. The project was tested in Eclipse and uploaded to GitHub.